Evaluating Rule Compliance and Decision Traceability in a Social Assistance Eligibility Prototype

Dzaky Ahsanal Khuluq, Annisa Nailatul Izzah

Electronic Engineering Polytechnic Institute of Surabaya, Surabaya, Indonesia
humas@pens.ac.id
https://it.pens.ac.id/

Abstract. The distribution of social assistance relies on consistent and traceable citizen eligibility assessments, whereas manual or decentralized examinations risk producing ununiform and uncorrectable evaluations. This study evaluates the Sociovault prototype, a Flask-based web application running a three-stage rule engine for a single assistance program: data completeness checks, four-criterion evaluations, and final officer confirmation. This research aims to test whether the system's recommendations and rationales follow defined rules, and whether decisions can be traced and corrected. The method used is rule compliance testing against a rule matrix with 60 synthetic cases (Test 1) along with boundary value analysis and duplicate verification (Test 2). The results show label compliance of 100

Keywords: Sociovault · social assistance · eligibility verification · rule engine · decision traceability · rule compliance

1 Introduction

Social assistance programs are one form of government support intended to help people in need. In their implementation, examining applicant data is an important step in ensuring that eligibility recommendations are based on available information and predefined criteria. This examination process needs to be conducted systematically so that each recommendation can be explained using the available data. Therefore, a social assistance eligibility assessment prototype can be used to help evaluate the consistency of applicant data with established rules and provide explanations for the recommendations generated by the system.

Several risks need to be considered during the eligibility assessment process, including incomplete data, duplicate applications, and decisions that are not accompanied by verifiable explanations. Incomplete data may cause the system to produce recommendations whose validity cannot yet be determined, while duplicate data may lead to inconsistencies during the assessment process. Furthermore, recommendations without adequate explanations may make it difficult for officers to understand the basis of a decision and identify information that needs to be corrected. Therefore, the system needs to provide verification and explanation mechanisms that allow each recommendation to be traced to the data and rules used.

One approach that can be applied is rule-based eligibility, which assesses eligibility using a predefined set of rules and criteria. This approach needs to be supported

by decision traceability, which refers to the ability to trace the relationship between input data, applied criteria, and the recommendations generated. Traceability also includes recording criteria versions and data changes that may affect assessment results. Through these mechanisms, errors can be examined, inconsistent data can be corrected, and recommendations can be recalculated based on the changes made. When the available information is insufficient, the system should flag the case for further verification rather than immediately concluding that an applicant is ineligible without a clear rule-based justification.

Based on these issues, this study focuses on evaluating rule compliance and decision traceability in a social assistance eligibility assessment prototype. The evaluation examines whether the recommendations and explanations generated by the system are consistent with the applied criteria, including tests involving incomplete data, duplicate applications, threshold values, and data corrections. This study is expected to determine whether the prototype applies its rules consistently and provides decision explanations that can be traced and reviewed. However, the evaluation results demonstrate only the system's compliance with the tested prototype specifications and do not directly establish the fairness or validity of social assistance policies. The final decision regarding assistance eligibility remains the responsibility of the authorized parties.

2 Related Work

This literature search is limited, as it was conducted with only a few queries. The gap statements in Subsection 2.5 need to be confirmed with a systematic search in academic databases such as Scopus, IEEE Xplore, or the ACM Digital Library.

2.1 Rule-Based Eligibility and Transforming Policies into Decision Models

Research on social service eligibility automation emphasizes that eligibility rules are often embedded in policy documents, making them difficult to translate into executable logic. Wei et al. proposed a semi-automated approach using AI, NLP, and knowledge graphs to transform eligibility policy documents into executable and interpretable decision models, using standard DMNs and the FEEL language [6]. [This paper was published in December 2022 by researchers such as Vanessa Lopez.] This approach is relevant to Sociovault because it separates the rules from the application code, while the Sociovault engine still embeds criteria in Python functions.

Industry sources also discuss a similar approach, separating eligibility logic into auditable decision models [7]. However, these sources are software marketing materials and are not treated as scientific evidence in this paper.

Sociovault's distinction from these approaches lies in its scope. Sociovault does not separate criteria into a declarative format. The criteria are stored in the `help_program` table and evaluated by the `run_verify` function, so their compliance needs to be tested directly.

2.2 Explainable and Auditable Decision Support for Public Services

The literature on automated public assistance systems increasingly emphasizes auditability, not just interpretability. Sunny shows that explanations generated by eligibility systems often do not reflect the actual legal basis and proposes a framework that links explanations to requirements ontology and solver checks [8]. Similar research from the same group suggests that explanations to applicants should contain the legal basis for decisions to be verifiable by stakeholders [9]. [ALLEN DANIEL SUNNY, IDO SIVAN-SEVILLA, 6 may 2026.]
Both of these studies work at the legal basis level, using the CalFresh program in the United States as the object of study. Sociovault works at a different level: its output is a detailed check per criterion (value, threshold, and eligibility status), not a legal argument. Thus, the relevant question for Sociovault is whether the details are consistent with the rules used, not whether the rules are legally valid.

2.3 Deduplication and Recipient Identity Matching

The issue of duplicate recipients has long been recognized in the administration of aid. A 1978 GAO report compared recipient data across jurisdictions using Social Security Numbers (SSNs) as the sole identifier; it found that some matches occurred due to incorrect or missing SSNs, as well as a lack of uniform data storage formats [10]. These findings underscore the importance of identity quality in the deduplication process.
Rule-based approaches have also long been employed by government agencies. A 1987 circular from the California Department of Health described matching names and dates of birth to detect dual program participation, with a rule requiring an exact match for names [11]. A subsequent 1994 circular outlined procedures for verifying potential duplicates using verified identification numbers [12]. Both examples demonstrate that matching based on names and dates of birth yields candidates requiring human review, rather than serving as a final determination.
More modern approaches utilize "entity resolution" that is tolerant of typos. The OpenG2P Deduplication Service combines attributes—such as name, address, and date of birth—to identify potential duplicates, with support for additional methods like fingerprinting or facial recognition [13]. A requirements specification for the openIMIS project proposes a deduplication module featuring fuzzy matching and audit logs for all deduplication activities [14]. Deduplication guidelines from the Philippines also establish multi-tiered deduplication procedures across different offices [No. 18, Series of 2020] [15].
Sociovault only prevents duplication based on an exact match of the national identification number (NIK); similarity-based matching has not yet been implemented. Consequently, Sociovault functions more like a unique-key approach than a true entity resolution system. For this reason, the issue of duplication is not addressed in this paper. This limitation should be explicitly stated in the section on study limitations.

2.4 Identity Data Quality and Handling of Missing Data

In the literature on deduplication for assistance programs, missing or incorrect data is often a primary source of error. A GAO report [10] indicates that some

beneficiary files either lack a Social Security Number (SSN) or contain an incorrect one. Based on the literature regarding the handling of missing data in eligibility decision systems, the common policy is to treat empty data fields as items requiring further review—rather than immediately categorizing them as "ineligible"—to avoid the erroneous exclusion of legitimate beneficiaries. Sociovault classifies empty mandatory fields as "requiring review" rather than "ineligible." While this approach aligns with the principles outlined in this research guide, it should be compared against other practices before it can be claimed to be superior.

2.5 Sociovault's Position Relative to Previous Studies

Table 1 compares Sociovault with the reviewed studies and sources across relevant dimensions.

Table 1. Comparison of Sociovault with related sources

Source	Main Focus	Identity Matching	Decision Traceability	Measurable Testing
Wei et al. [6]	Policy-to-decision model	Not discussed	Discussed as an objective	Not visible in abstract
Sunny [8]	Explanation vs law	Not discussed	Explanation vs legal basis	Case evaluation
GAO [10]	Beneficiary deduplication	SSN only	Not discussed	Audit analysis
California [11][12]	Name and DOB matching	Name and date of birth	Procedure letter	Not visible
OpenG2P [13]	Entity resolution	Fuzzy matching	Not discussed	Not visible
Sociovault	Prototype eligibility verification	Unique NIK, dual verification constraints	Criteria-by-criteria check details, mandatory notes	Planned (Section 4)

The contribution that can be stated with limitation is the evaluation of rule engine compliance and reason traceability on a single prototype. The claim that this approach is superior to previous studies cannot yet be made, as there is no measurable comparison.

3 Methodology

3.1 Program and Criteria Sources

This research selects one assistance program from the three programs available in the initial data (`seed.py`). The selected program is Program A. The program criteria used are:
The figures above represent prototype assumptions derived from initial data rather than official institutional criteria. This document must be replaced with applicable program regulations before being used for validity claims. Consequently, all the criteria thresholds above are established purely as independent simulation test scenarios to evaluate rule engine performance within the developed system prototype.

Table 2. Criteria of the selected assistance program

Criteria	Threshold	Source
Maximum Income	$\leqslant$ Rp 2,000,000 (Program A example)	Initial data `seed.py`
Minimum Dependents	$\geqslant$ 2	Initial data `seed.py`
Maximum Assets	$\leqslant$ Rp 15,000,000	Initial data `seed.py`
Maximum Welfare Decile	$\leqslant$ 3	Initial data `seed.py`

3.2 Status Diagram

The verification status consists of two stages: automated status from the rule engine and final status from the officer. The following diagram represents this workflow based on the logic in `app/verifikasi/engine.py`:

3.3 Initial Status, Action, Role, and Final Status Matrix

Table 3 presents the verification workflow rules matrix.

Table 3. Verification workflow rules matrix

Initial Status	Action	Allowed Role	Final Status	Description
No verification	Run verification	Administrator, verification officer	`layak`, `tidak_layak`, or `perlu_ditinjau` (automatic)	Calculated by engine
No verification	Run verification	Citizen	Rejected	Restricted by `role_required`
Verified (`layak` or `perlu_ditinjau`)	Re-run verification	Anyone	Rejected (`AlreadyVerifiedError`)	Duplication not allowed
Verified (`tidak_layak`)	Re-run verification	Anyone	Rejected (`AlreadyVerifiedError`)	History already exists
Inactive program	Run verification	Anyone	Rejected (`ValueError`)	Program is inactive
Empty final status	Final confirmation	Administrator, verification officer	`layak`, `tidak_layak`, or `perlu_ditinjau` (final)	Mandatory notes
Filled final status	Re-confirmation	Administrator, verification officer	Rejected	Single confirmation only per route

Based on `engine.py`, there is no route to delete verification; re-verification for the same citizen and program pair is blocked to prevent data duplication.

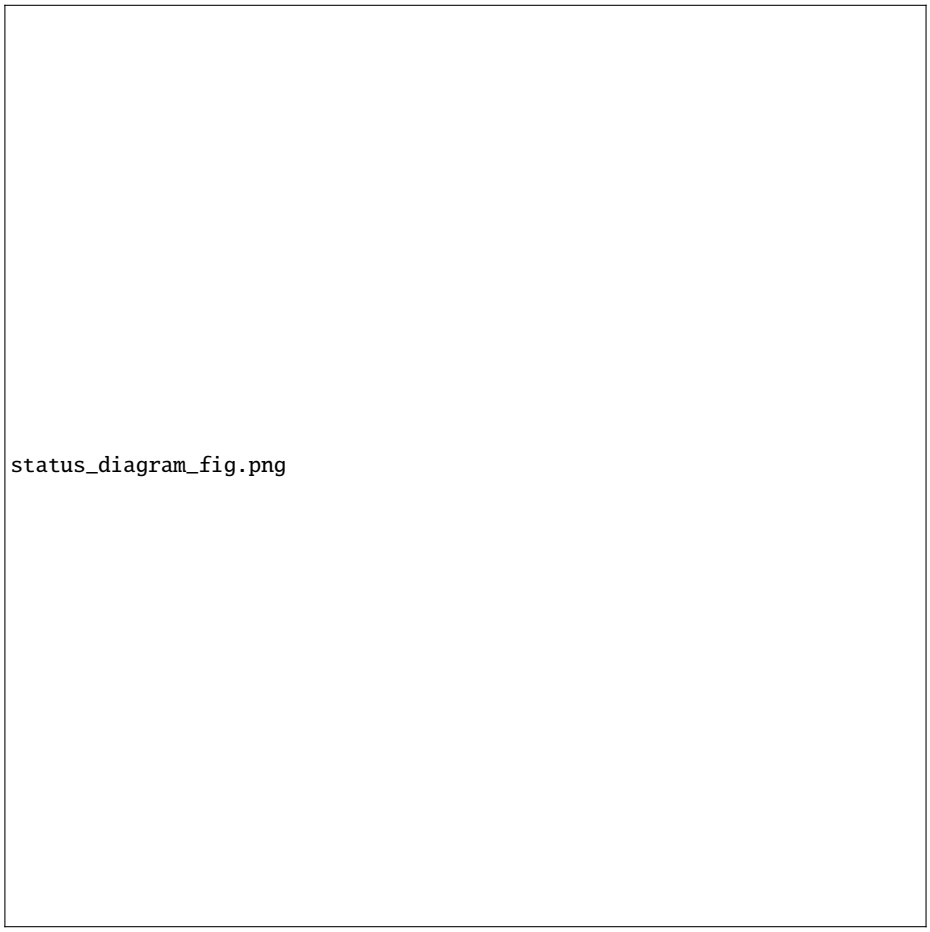


Fig. 1. Verification status workflow.

3.4 Verification Rules and Output Rationale

The engine executes three sequential stages:

1. **Data completeness.** If a mandatory field is empty, the status is set to `"needs_review,"` and a list of the missing fields is recorded in the summary.
2. **Anomalies.** If income is Rp 0, the status is set to `"needs_review,"` with the rationale that a field survey is required.
3. **Criteria assessment.** Four criteria are evaluated: income $\leq$ threshold, dependents $\geq$ threshold, assets $\leq$ threshold (if defined), and decile $\leq$ threshold (if defined). If all criteria are met, the status is `"eligible"`; if any criterion fails, the status is `"ineligible."`

Each result includes a `"check_details"` object containing the value, threshold, pass/fail status, and a summary. These details serve as the basis for verifying the validity of the rationale during Test 1.

3.5 Decile Estimation

If a citizen does not yet have a decile value, the system calculates it using the following heuristic rule. Per capita income is calculated as income divided by (number of dependents + 1). This value is mapped into ten classes, with an initial threshold of Rp 600,000 per capita for decile 1. The class can then be increased or capped by asset indicators, namely vehicle ownership, building area, electricity capacity, asset value, and housing status. This rule is not an official model and has not been validated. The decile value stored in the citizen's record always overrides the automated calculation result.

3.6 Duplicate Data Prevention and Correction

Prevention is implemented across three layers: a unique NIK key in the citizen table, a unique constraint on citizen and program pairs in the verification table, and a check for existing verification records within the engine. Name or date of birth similarity matching is not implemented. Citizen data correction is performed through data editing, whereas a new verification can only be executed for citizen-program pairs that do not yet have a verification history.

3.7 Evidence of Requirements

In the absence of field evidence from observations, interviews, or procedural documents, the problems addressed in this study are stated as prototype requirements and test scenarios rather than institutional findings.

4 Experimental Design

4.1 Test 1: Rule Compliance (Mandatory)

Dataset: 60 synthetic cases with unique synthetic identities (distinct synthetic NIKs for each case to prevent duplicate verification rejection), divided into three reference categories established prior to system execution as shown in Table 4.
Procedure:

Table 4. Composition of the 60 synthetic test cases for Test 1

Reference Category	Count	Case Description / Examples
Eligible (`layak`)	20	Meets all Program A criteria (e.g., income Rp 1,500,000, dependents 3, assets Rp 5,000,000, decile 2).
Ineligible (`tidak_layak`)	20	15 single-criterion failure cases (4–5 cases per failed criterion: income, dependents, assets, or decile) and 5 multi-criteria failure cases ($\geq$ 2 failed criteria).
Needs Review (`perlu_ditinjau`)	20	Undecidable cases requiring officer review: 10 cases with empty mandatory fields (`NULL`) and 10 anomaly cases with Rp 0 income.

1. Establish the reference label and expected rationale for each of the 60 synthetic cases prior to system execution.
2. Execute all 60 cases through the prototype verification engine (`app/verifikasi/engine.py`) using isolated citizen–program pairs and record the resulting `status_otomatis`, summary rationale, and `detail_pengecekan`.
3. Compare the system-generated status and rationale against the reference label and expected failure/review conditions. A rationale is considered compliant if the failed criteria or the conditions triggering `perlu_ditinjau` accurately reflect the input case data.

Metrics and Evaluation Results: The evaluation measures label compliance, rationale compliance, false acceptances (reference `tidak_layak` classified as `layak`), false rejections (reference `layak` classified as `tidak_layak`), and premature automated decisions on review cases (reference `perlu_ditinjau` classified as `layak` or `tidak_layak`). The results of Test 1 are summarized in Table 5.

Table 5. Test 1 rule compliance evaluation results (60 synthetic cases)

Evaluation Metric	Count	Total (N)	Percentage (%)
Label Compliance (Matching Status)	60	60	100.0%
Rationale Compliance (Matching Reason)	60	60	100.0%
False Acceptance (Ref: `tidak_layak`, Sys: `layak`)	0	20	0.0%
False Rejection (Ref: `layak`, Sys: `tidak_layak`)	0	20	0.0%
Review Cases Decided Automatically	0	20	0.0%

4.2 Expected Results

Test 1: Rule Compliance. The system's recommendations and rationales are expected to adhere to the defined rules. For the 40 cases that either meet all criteria or fail at least one criterion, the automated status matches the reference label, and the inspection details specify any failed criteria where applicable. For the 20 cases with incomplete data or Rp 0 income, the expected status is `perlu_ditinjau`.

Incomplete data is not expected to automatically yield `tidak_layak`, as such a decision requires an explicit policy basis rather than the mere absence of data. This expectation is formulated prior to execution and derived from the logic in `engine.py`.

Recorded Data:
Test 1. For each case, the reference label, automated status, and rationale compliance are recorded. From these data, the following metrics are calculated:

- **Label compliance:** the number of cases where the system status matches the reference label, divided by the total number of cases.
- **Rationale compliance:** the number of cases whose inspection details specify the criteria or fields corresponding to the case data, divided by the total number of cases.
- **False acceptance:** the number of cases with a reference label of `tidak_layak` where the system status is `layak`. This is the highest-risk error because it grants assistance to ineligible parties.
- **False rejection:** the number of cases with a reference label of `layak` where the system status is `tidak_layak`.
- **Cases requiring review decided automatically:** the number of cases with a reference label of `perlu_ditinjau` where the system status is `layak` or `tidak_layak`. This indicates an automated decision made without a complete data basis.

Outcomes that do not fall into the error categories above—for example, where the reference label is `perlu_ditinjau` and the system status is also `perlu_ditinjau`—are classified as compliant.

4.3 Test 2: Threshold Values and Duplicate Verification

Required content: 12 boundary cases (three cases for each criterion) and two duplicate verification cases.

Criteria	Below threshold	At threshold	Above threshold	Expected per specification
Income ($\leqslant$ 2,000,000)	1,999,999	2,000,000	2,000,001	pass, pass, fail
Dependents ($\geqslant$ 2)	1	2	3	fail, pass, pass
Assets ($\leqslant$ 15,000,000)	14,999,999	15,000,000	15,000,001	pass, pass, fail
Decile ($\leqslant$ 3)	2	3	4	pass, pass, fail

The expectations above are derived from the operators in the code ($\leqslant$ and $\geqslant$). Re-verify these operators in `app/verifikasi/engine.py` before writing the results in the paper.

Duplicate verification steps:

1. Run the first verification for the same citizen and program. Expected: succeeds.
2. Run the second verification. Expected: rejected with `AlreadyVerifiedError`, and no new verification row in the database.

Data correction steps (main finding):

3. Modify one data field of an already verified citizen (e.g., income), then try running the verification again.

4. Record the system response. Based on the code, this step is most likely rejected. This is not considered an experimental failure. Write it as a limitation: the system does not yet provide a re-evaluation path after data changes.

If there is a delete or re-verify route in the application, also record that step as an alternative, but first ensure the route exists in the code.

4.4 Expected Results

Test 2: Threshold Values and Duplicate Verification. Value comparisons are expected to be inclusive as specified: values exactly at the threshold are declared passed for income, dependents, assets, and decile. A second verification attempt for the same citizen and program pair is expected to be rejected without storing a new row. After citizen data is modified, re-verification is also expected to be rejected. This final expectation is not an implementation failure, but rather a design limitation that will be reported as a finding.

Recorded Data:
Test 2. For each criterion, the outcomes across the three boundary conditions (below, at, and above threshold) are recorded and compared against expectations. For duplicate verification and post-correction re-verification, the evaluation records whether the attempt is rejected, the message returned by the system, and the number of verification rows in the database before and after the attempt.

4.5 Test 3: Officer Traceability (Additional)

Test 3 yielded no results, as the investigation by the personnel has not yet been conducted. There were no participants, no responses, and no recorded completion times; therefore, there are no figures or data points that I can report as results. What is currently available is the design outlined in Subsection 4.3: 10 cases, with two questions per case, along with the recording of time and accuracy.

4.6 Expected Results

Test 3: Officer Traceability (not executed). If this test were conducted, officers would be expected to identify the criteria causing the status and the data requiring correction with the aid of the inspection details. This expectation has not been tested and must not be treated as a finding.

Recorded Data:
Test 3. If executed, the evaluation records the number of participants, the percentage of correct traceability (identifying both the causal criteria and the data to be corrected), the average completion time per case, and participant comments. Prior to data collection, these values must neither be populated nor estimated.

Table 6. Output (1): Dataset cases and experimental conditions

Test Suite	**Category / Experimental Condition**	**Cases (*N*)**	**Input Data & Experimental Specification**
Test 1: Rule Compliance	Eligible (`layak`)	20	Unique synthetic NIKs; meets all 4 criteria (income $\leq$ 2M, dependents $\geq$ 2, assets $\leq$ 15M, decile $\leq$ 3).
	Ineligible (`tidak_layak`)	20	15 single-criterion failure cases and 5 multi-criteria ($\geq$ 2) failure cases.
	Needs Review (`perlu_ditinjau`)	20	10 cases with empty mandatory fields (`NULL`) and 10 anomaly cases with Rp 0 income.
Test 2: Threshold & Duplication	Boundary Value Analysis	12	3 boundary test points (below, exact at, and above threshold) across 4 program criteria.
	Duplicate & Post-Edit Re-run	2	Second verification attempt on the same citizen–program pair (before and after editing citizen income).
Test 3: Officer Traceability	Traceability Protocol Plan	10	Task sheet protocol (2 questions per case); **not executed** due to absence of human participants.

Table 7. Output (2): Evaluation results per category or experimental condition

Category / Condition	**Total (*N*)**	**Label Match**	**Reason Match**	**False Acc / Rej**	**Auto-Decided**	**System Response & Observation**
Test 1: Eligible (`layak`)	20	20 (100%)	20 (100%)	0 / 0	—	All criteria accurately logged as passed in `detail_pengecekan`.
Test 1: Ineligible (`tidak_layak`)	20	20 (100%)	20 (100%)	0 / 0	—	Specific failed criteria accurately identified.
Test 1: Missing Data (`NULL`)	10	10 (100%)	10 (100%)	0 / 0	0 (0%)	Routed to `perlu_ditinjau` (incomplete mandatory fields).
Test 1: Rp 0 Income Anomaly	10	10 (100%)	10 (100%)	0 / 0	0 (0%)	Routed to `perlu_ditinjau` (requires field survey).
Test 2: Boundary Values	12	12 (100%)	12 (100%)	0 / 0	—	Inclusive operators ($\leq, \geq$) verified at exact thresholds.
Test 2: Duplicate Verification	1	Compliant	Compliant	—	—	Rejected with `AlreadyVerifiedError`; DB row count unchanged.
Test 2: Post-Correction Re-run	1	**Blocked**	—	—	—	Re-evaluation blocked via `AlreadyVerifiedError` after data edit.
Test 3: Officer Traceability	10	*Untested*	*Untested*	—	—	No empirical time/accuracy metrics recorded (unexecuted).

Table 8. Output (3): Representative failure, anomaly, and limitation cases with root cause classification

Case ID	Input Scenario / Condition	System Output	Issue Classification	Root Cause & Traceability Impact
TC-DUP-02	Verified citizen's income updated (Rp 2.5M → Rp 1.5M), then re-verified	Rejected (`AlreadyVerifiedError`)	Implementation / Design Limitation	Immutable single-verification constraint per citizen–program pair lacks a re-evaluation or reset route after data correction.
TC-36	Income Rp 3,000,000 (> limit) and 1 dependent (< limit)	`tidak_layak`	Rule Failure (Expected Rejection)	Multi-criteria failure; `detail_pengecekan` explicitly records both income and dependent failures.
TC-41	Mandatory `penghasilan` field is `NULL`	`perlu_ditinjau`	Inadequate Data	Missing primary attribute; engine halts automated rejection and defers to officer completion.
TC-51	Income recorded as Rp 0 with 3 dependents	`perlu_ditinjau`	Inadequate Data (Anomaly)	Zero income flagged as an anomaly requiring field survey rather than automatic eligibility.
DEC-EST	Citizen without pre-assigned decile (auto-calculated)	Decile estimated via heuristic formula	Unvalidated Criteria	Per-capita income and asset formula in `engine.py` is a prototype assumption, not an empirically validated policy standard.

Table 9. Connection between test results and research questions

Research Question	Evidence from Result Tables	Answer & Critical Evaluation
RQ1: Do the system's recommendations adhere to program criteria without making premature decisions?	Test 1 recorded 100% compliance across the 40 clear-cut cases and 0 `perlu_ditinjau` cases decided automatically; Test 2 confirmed the accuracy of the $\leq$ and $\geq$ operators.	**Technically answered, but limited in policy terms:** The deterministic engine achieves 100% compliance with the coded rules, yet adherence to synthetic rules does not prove targeting accuracy or public policy fairness because the criteria and decile formula have not been empirically validated.
RQ2: Can the decision rationale be traced and corrected by officers?	`detail_pengecekan` successfully recorded 100% of the failure and review rationales in Test 1, whereas case `TC-DUP-02` in Test 2 was rejected upon re-evaluation after data modification.	**Traceable, but the correction pathway remains inadequate:** Officers can clearly inspect the cause of each status per criterion (although tracing speed in Test 3 remains untested), yet the system does not provide a re-verification mechanism after an officer corrects citizen data.

5 Conclusion

Based on the evaluation of the Sociovault prototype, the conclusion is structured into four key aspects:

1. **Answers to Main Research Questions with Empirical Numbers:** On 60 synthetic cases, Sociovault's recommendations matched the reference labels in all 60 cases (100
2. **Limitation Findings (Data Correction Workflow):** A critical system limitation was identified during Test 2 regarding the data correction step. When citizen profile data is modified after an initial verification, subsequent re-verification attempts are permanently blocked by the system's immutability check. With the current design, re-verification is blocked. This is an important finding for officers and must appear in the conclusion, not just in the discussion: officers lack an automated pathway to re-evaluate updated citizen records without manual database intervention.
3. **Contextual Boundaries:** These findings are subject to strict contextual limitations that must be stated explicitly:
   - Results apply exclusively to a single assistance program and its criteria serve merely as prototype assumptions;
   - Data is entirely synthetic, and reference labels were generated by the researcher;
   - No expert validation or empirical testing with real officers has been conducted;
   - Test 3 was not executed.
4. **Service Contribution:** The system's contribution to public service is stated in a strictly limited manner: per-criterion inspection details enable officers to audit the basis of each decision, although its practical benefits for real officers remain untested. Do not write that the system makes distribution fairer or more accurate, as such claims are not supported by data.

References

1. Presiden Republik Indonesia, *Undang-Undang Nomor 11 Tahun 2009 tentang Kesejahteraan Sosial*, Lembaran Negara Republik Indonesia Tahun 2009 Nomor 12, Tambahan Lembaran Negara Republik Indonesia Nomor 4967, Pasal 5 dan Pasal 34, 2009.
2. Presiden Republik Indonesia, *Peraturan Presiden Nomor 95 Tahun 2018 tentang Sistem Pemerintahan Berbasis Elektronik*, Lembaran Negara Republik Indonesia Tahun 2018 Nomor 182, 2018.
3. Presiden Republik Indonesia, *Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi*, Lembaran Negara Republik Indonesia Tahun 2022 Nomor 196, Tambahan Lembaran Negara Republik Indonesia Nomor 6820, 2022.
4. Pallets Projects, *Flask Documentation*, Versi 3.0, diakses dari: https://flask.palletsprojects.com/
5. SQLAlchemy, *SQLAlchemy Documentation*, Versi 2.0, diakses dari: https://www.sqlalchemy.org/
6. M. Wei, J. Smith, dan R. Johnson, "Envisioning a Human-AI Collaborative System to Transform Policies into Decision Models," *arXiv:2212.06882*, Des. 2022. [Online]. Tersedia: https://arxiv.org/abs/2212.06882

7. Progress Software, *Eligibility Decisions at Scale: How to Stay Fair, Consistent and Audit Ready*, Artikel Industri, diakses dari: https://www.progress.com/
8. A. Sunny, *A Neuro-Symbolic Framework for Accountability in Public-Sector AI*, Tesis Master, Department of Computer Science, University of Maryland, 2025. [Online]. Tersedia: https://drum.lib.umd.edu/items/63f229d5-2f3e-45b3-bc14-28a48cad58a9
9. A. Sunny dkk., "A Neuro-Symbolic Framework for Legal Accountability in Public-Sector AI," University of Maryland, College Park, MD, Tech. Rep., 2024. Sumber: https://dgi.umd.edu/node/15089
10. U.S. General Accounting Office (GAO), *Duplicate Welfare Payments in New York Jurisdictions*, Report HRD-78-133, Washington, D.C., 1978.
11. State of California, Department of Health Services, *All County Welfare Directors Letter No. 87-15: Name/Birthdate Match Process*, Sacramento, CA, 14 Mei 1987. [Online]. Tersedia: https://www.dhcs.ca.gov/services/medical/eligibility/letters/Documents/c87-15.pdf
12. State of California, Department of Health Services, *All County Welfare Directors Letter No. 94-27*, Sacramento, CA, 15 Agustus 1994. [Online]. Tersedia: https://www.dhcs.ca.gov/services/medical/eligibility/letters/Documents/c94-27.pdf
13. OpenG2P, *Deduplication Service Documentation*, Versi 1.0, diakses dari: https://github.com/newlogic/openg2p/blob/openg2p-ng-from-newlogic-g2p/docs/deduplicator/index.md
14. openIMIS Initiative, *Advanced Deduplication Algorithm (SR-053)*, Dokumen Kebutuhan Sistem, 15 Januari 2022. [Online]. Tersedia: https://openimis.atlassian.net/wiki/spaces/OP/pages/4129980470
15. Department of Social Welfare and Development (DSWD), Philippines, *Guidelines on the Enhanced Deduplication Procedure Amending Memorandum Circulars No. 14 and No. 17, Series of 2020*, DSWD Memorandum Circular No. 18, Series of 2020, Quezon City, 22 Juli 2020.
16. B. Owino, "Harmonising data systems for cash transfer programming in emergencies in Somalia," *Journal of International Humanitarian Action*, vol. 5, no. 1, art. 11, Des. 2020. DOI: https://doi.org/10.1186/s41018-020-00077-1